

יחידה 7.3 – רשימות

פעולות מתקדמות ותבנית הצבירה



חזרה קצרה

append, pop, sort – ואיך הרשימה נראית אחרי כל אחת.

append



5 ↗ 6

Total: 6

pop



5 ↘ 4

Total: 4

sort



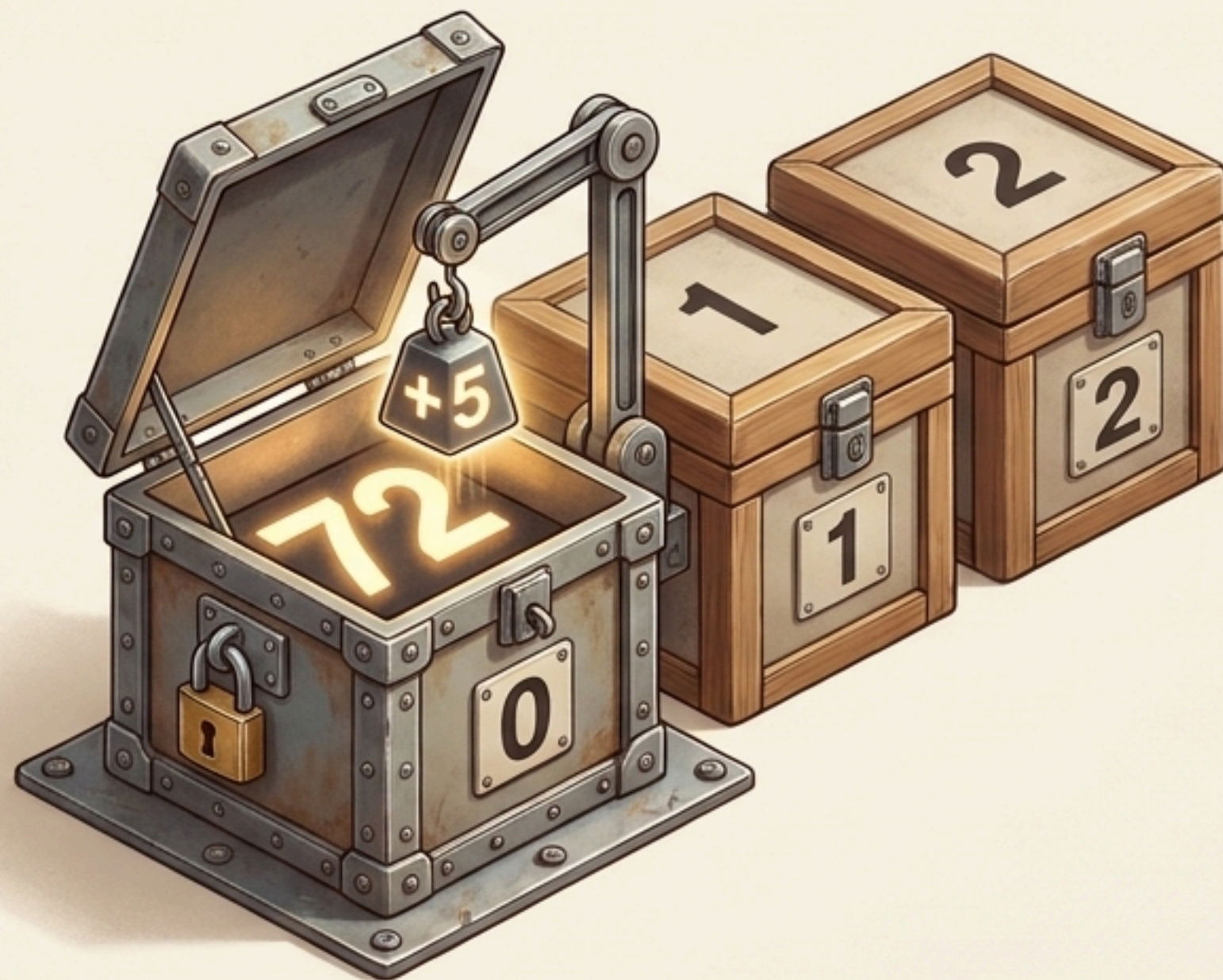
5 ↻

Total: 5 (Reordered)

עדכון לפי אינדקס בתוך לולאה

```
scores = [72, 84, 65]
for index in range(len(scores)):
    scores[index] += 5

print(scores)    # [77, 89, 70]
```



כשצריך לשנות ערך במקומו ברשימה, לולאת אינדקס (`range(len(...))`) היא הכלי הנכון — לא לולאה ישירה על הערכים.

למה לא להסיר ככה?

```
values = [2, 4, 6, 8, 10]
for value in values:
    if value % 2 == 0:
        values.remove(value)
```



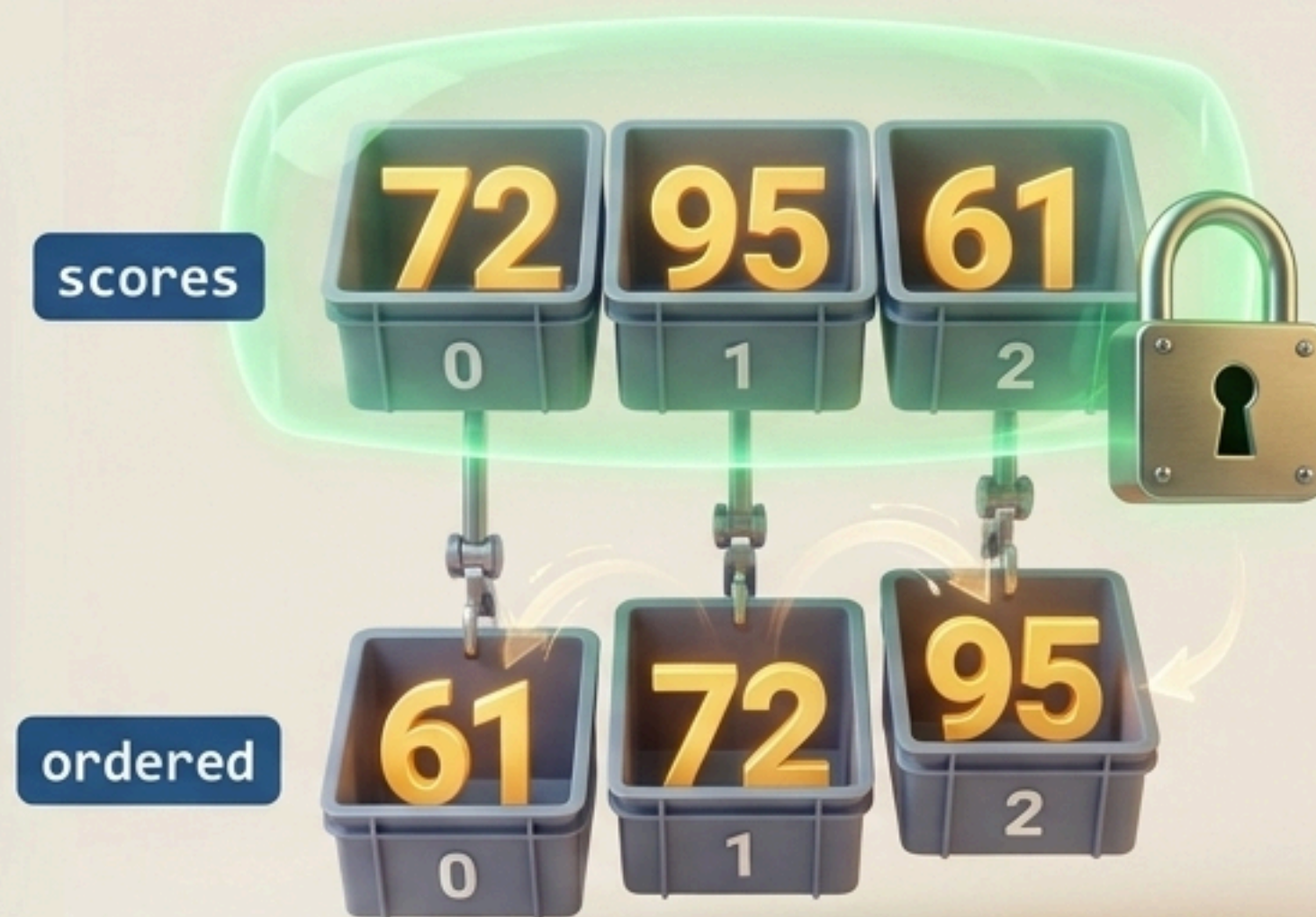
הסרה משנה את המיקומים של כל הפריטים שאחרי – הלולאה ממשיכה להתקדם בלי לדעת שהמיקומים זזו, וכמה פריטים "מדלגים" בטעות.
תיקון: לבנות רשימה חדשה של מה לשמור, במקום להסיר מהרשימה שעליה לולאה.

מעתיקים לפני שממיינים



```
scores = [72, 95, 61]
ordered = scores.copy()
ordered.sort()
```

```
print(scores)      # [72, 95, 61] - נשאר כמו שהיה
print(ordered)     # [61, 72, 95]
```



`.copy()` (או `[:]`) לפני `.sort()`. כאשר גם הסדר המקורי צריך לשרוד.

הצורך בצבירה ותבנית ארבעת השלבים

```
def calculate_total(values):
```

```
    total = 0
```

1. אתחול לפני הלולאה

```
    for value in values:
```

2. מעבר

```
        total += value
```

3. עדכון

```
    return total
```

4. שימוש בתוצאה אחרי הלולאה



במקום משתנה נפרד לכל פריט – סכום מצטבר אחד, בונים אותו צעד אחר צעד.

מעבר למעבדה



היום: לא להסיר
להסיר תוך כדי מעבר
ישיר על הרשימה,

ותבנית הצבירה
בארבעה צעדים.



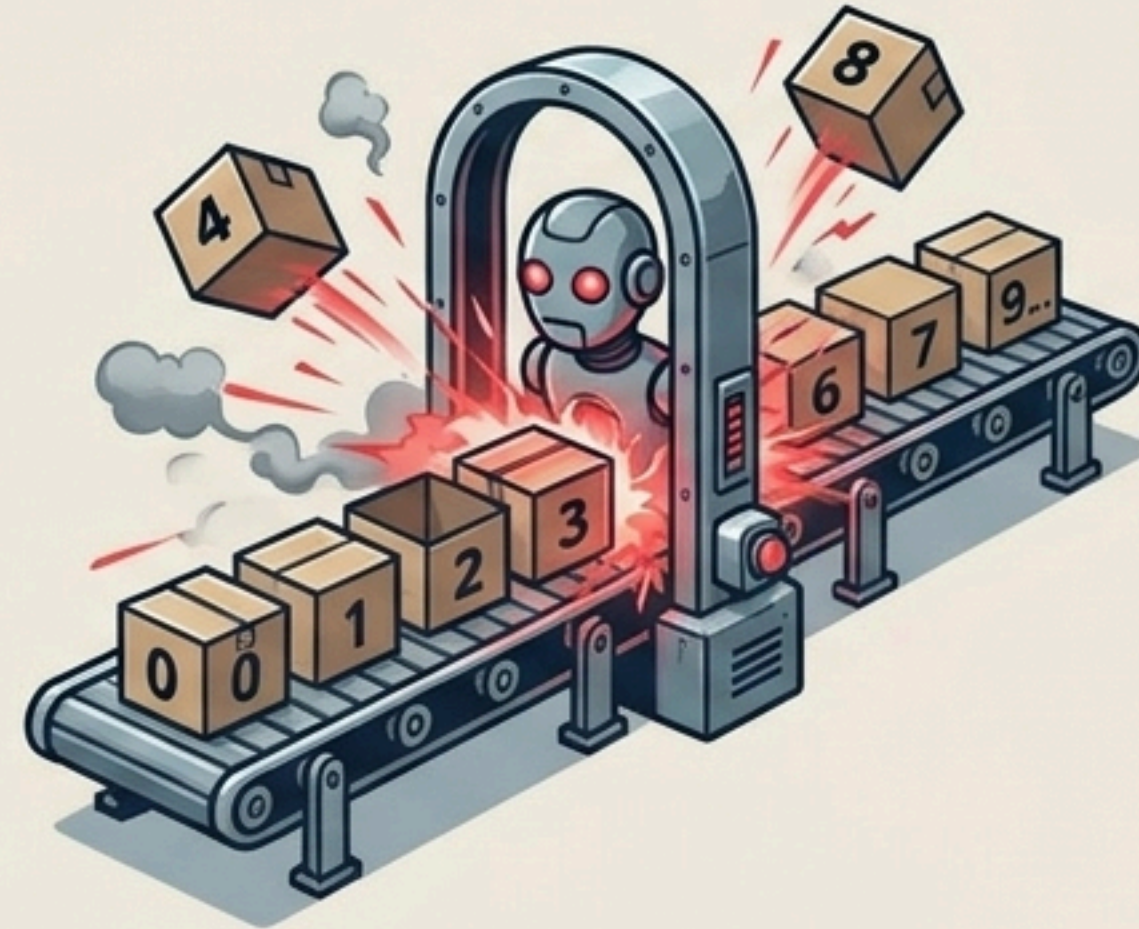
חימום: מדלגים בגלל ההזזה

```
values = [2, 4, 6, 8, 10]
for value in values:
    if value % 2 == 0:
        values.remove(value)
print(values)
```

חזו [] (כל הערכים זוגיים), ואז הריצו.

חימום: התשובה

[4, 8]



שני ערכים "חמקו". הסרה מזיזה כל פריט מאוחר יותר מקום אחד שמאלה, אבל מיקום הלולאה ממשיך להתקדם בלי קשר – חלק מהפריטים אף פעם לא נבדקים. תיקון: לבנות רשימה חדשה של מה לשמור.



משימה 1:

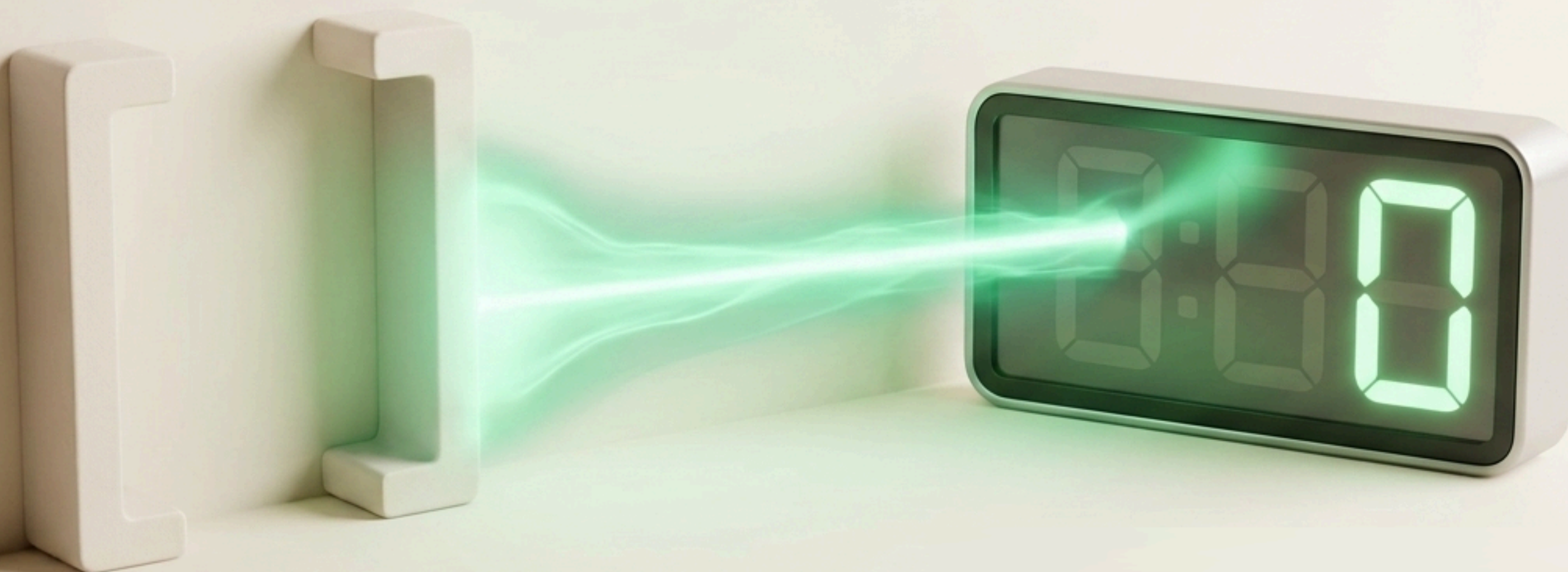
`get_sorted_copy(values)`



מחזירה רשימה חדשה ממוינת, בלי לגעת במקור.
בדקו על [61, 95, 72] ואשרו שהמקור עדיין [61, 95, 72] אחרי כן.



משימה 2: calculate_total ו-
calculate_average



calculate_average מחזירה 0 עבור רשימה ריקה (אף פעם לא לחלק באפס).
בדקו את שתיהן על [65, 84, 72].



שומרים + בדיקת יציאה

שומרים: G8_U7_M3_Accumulate_<Name>.py

1. מהו הערך ההתחלתי של סכום?
2. איפה return נמצא בפונקציית סכימה – בתוך הלולאה, או אחריה?
3. למה בודקים רשימה ריקה לפני חישוב ממוצע?
4. איך יוצרים עותק ממזין בלי לפגוע בסדר המקורי?

בדיקת יציאה: תשובות לדוגמה

1. `0` – לפני הלולאה, פעם אחת.
2. אחרי הלולאה – בתוכה הוא יחזיר אחרי איטרציה אחת בלבד.
3. כדי למנוע `ZeroDivisionError` בחלוקה ב-`len(values)`.
4. `copy()` או `[:]` לפני `sort()`.